

# TP2 - Computacao Grafica: Curvas Parametricas

Aluno: Antonio Neto

## 1. Objetivo

Este trabalho estende a aplicacao desenvolvida no TP1 com a implementacao de dois tipos de curvas parametricas, de forma interativa:

- Curva de Bezier (algoritmo de De Casteljau);
- Curva B-Spline cubica uniforme (forma matricial).

O codigo dos dois algoritmos foi obtido de implementacoes prontas de referencia e adaptado a estrutura do projeto. As fontes estao indicadas na Secao 8.

## 2. Organizacao do codigo

A aplicacao e escrita em Python com a biblioteca Tkinter. As responsabilidades sao divididas em modulos:

| Arquivo            | Responsabilidade                                   |
|--------------------|----------------------------------------------------|
| main.py            | Interface grafica, eventos de mouse e integracao.  |
| models.py          | Classes das primitivas, incluindo a classe Curva.  |
| rasterization.py   | Rasterizacao de retas e circunferencia.            |
| transformations.py | Transformacoes geometricas 2D.                     |
| clipping.py        | Recorte de retas (Cohen-Sutherland, Liang-Barsky). |
| curvas.py          | Geracao das curvas Bezier e B-Spline.              |

Em relacao ao TP1, foram acrescentados o modulo curvas.py, a classe Curva em models.py e a integracao na interface (main.py).

A classe Curva armazena a lista de pontos de controle e o tipo da curva (BEZIER ou BSPLINE). As funcoes de curvas.py recebem os pontos de controle e retornam uma lista de pontos amostrados sobre a curva, no formato [(x, y), ...].

## 3. Funcionamento na interface

O usuario seleciona o modo "Curva Bezier" ou "Curva B-Spline" e clica no canvas para definir os pontos de controle. Ao clicar em "Finalizar Curva", a curva e gerada. A curva de Bezier exige no minimo 2 pontos de controle; a B-Spline cubica exige no minimo 4.

O metodo `_desenhar_objeto` percorre os pontos amostrados retornados por curvas.py e liga pontos consecutivos com segmentos de reta, utilizando o algoritmo de Bresenham ja existente. A curva continua e, portanto, aproximada por uma sequencia de pequenos segmentos.

As curvas tambem respondem as transformacoes geometricas (translacao, rotacao, escala e reflexao), aplicadas aos pontos de controle, e a selecao por area retangular.

## 4. Algoritmo de De Casteljau (Bezier)

Uma curva de Bezier de grau  $n$  e definida por  $n+1$  pontos de controle. Para cada parametro  $t$  em  $[0, 1]$  corresponde um ponto da curva; variando  $t$  de 0 a 1 obtem-se a curva completa. A curva inicia no primeiro ponto de controle e termina no ultimo.

O algoritmo de De Casteljau calcula o ponto correspondente a um dado  $t$  por interpolacoes lineares

sucessivas:

- Para cada par de pontos vizinhos, calcula-se um ponto intermediario:  $P' = (1 - t) * P_i + t * P_{(i+1)}$ ;
- O processo gera um conjunto com um ponto a menos e e repetido nivel a nivel;
- Quando resta um unico ponto, este e o ponto da curva para aquele t.

A funcao `bezier_de_casteljau` (em `curvas.py`) aplica este procedimento para um conjunto de valores de t (200 amostras por padrao) e funciona para qualquer numero de pontos de controle.

## 5. B-Spline cubica uniforme

A curva B-Spline nao passa pelos pontos de controle e possui controle local: a alteracao de um ponto de controle afeta apenas a regio vizinha da curva.

Na versao implementada, cada segmento da curva e definido por quatro pontos de controle consecutivos ( $p_0, p_1, p_2, p_3$ ). Para uma lista com n pontos, sao gerados  $n - 3$  segmentos, deslocando uma janela de quatro pontos ao longo da lista. Por isso, sao necessarios no minimo 4 pontos de controle.

Cada segmento e avaliado, para t em  $[0, 1]$ , pelas funcoes de mistura da B-Spline cubica uniforme (derivadas da matriz base, fator  $1/6$ ):

$$\begin{aligned} B_0(t) &= (-t^3 + 3t^2 - 3t + 1) / 6 \\ B_1(t) &= (3t^3 - 6t^2 + 4) / 6 \\ B_2(t) &= (-3t^3 + 3t^2 + 3t + 1) / 6 \\ B_3(t) &= (t^3) / 6 \end{aligned}$$

$$P(t) = B_0(t) * p_0 + B_1(t) * p_1 + B_2(t) * p_2 + B_3(t) * p_3$$

A funcao `bspline_uniforme_cubica` (em `curvas.py`) percorre os grupos de quatro pontos, amostra t em cada segmento e concatena os resultados em uma unica lista de pontos.

## 6. Recorte de curvas

Os algoritmos de recorte (Cohen-Sutherland e Liang-Barsky) originalmente atuavam apenas sobre retas. Como a curva e desenhada como uma sequencia de segmentos, o recorte foi estendido a ela: ao recortar, a janela e armazenada na curva e, no desenho, cada segmento e recortado individualmente pelas mesmas funcoes de `clipping.py`. Assim, apenas a porcao da curva interna a janela e exibida.

## 7. Comparacao entre as curvas

| Caracteristica       | Bezier       | B-Spline cubica    |
|----------------------|--------------|--------------------|
| Passa pelos extremos | Sim          | Nao                |
| Minimo de pontos     | 2            | 4                  |
| Tipo de controle     | Global       | Local              |
| Calculo              | De Casteljau | Funcoes de mistura |

## 8. Fontes do codigo adaptado

- Algoritmo de De Casteljau: "De Casteljau's algorithm", Wikipedia ([en.wikipedia.org/wiki/De\\_Casteljau's\\_algorithm](http://en.wikipedia.org/wiki/De_Casteljau's_algorithm)).
- B-Spline cubica uniforme (forma matricial): D. Hearn e M. P. Baker, "Computer Graphics with OpenGL", e materiais correlatos sobre B-Splines uniformes.

O codigo dessas referencias foi adaptado para o formato de pontos (x, y) utilizado no projeto e integrado a

interface Tkinter do TP1.